CS123 PROGRAMMING ASSIGNMENT

LEVENSHTEIN DISTANCE

OVERVIEW

This program focuses on refreshing your programming with Java Collections classes. You will implement a module that finds a simplified Levenshtein distance between two words represented by strings.

Your program will need support files `LevenDistanceFinderMain.Java`, and `dictionary.txt,` while you will be implementing your code in the `LevenProcessor.java` file.

INSTRUCTIONS

The Levenshtein distance, named for its creator Vladimir Levenshtein, is a measure of the distance between two words. The edit distance between two strings is the minimum number of operations that are needed to transform one sting into the other. For a full implementation of the distance, we would need to account for three different types of operations, removing a letter, adding a letter, or changing a letter.

For THIS program however, we are going to focus solely on the ability to change one letter to another. So, we will be thinking of changing one letter at a time, while maintaining the fact that the word is a still a valid word at all times.

So, for an example, the Levenshtein distance from "dog" to "cat" is 3. One possible path is

```
"dog" to "dot" to "cot" to cat"
```

Note, that there might be other paths, but they all involve at least 3 changes. Another longer example is from the word "witch" to the word "coven".

```
witch->watch->match->march->marcs->mares->mores->moves->coves->coven
```

You will be implementing a secondary class called `LevenshteinProcessor` that will be used by the main program to solve the problem.

SAMPLE EXECUTIONS

This is what the program should like when you run it.

```
* Welcome to the Levenshtein Edit distance solver                *
* Please type in two words of the same size that will be used   *
* To find a path between them.                                   *
*****************************************************************

What word would you like to start with?
--->Witch
What word would you like to end with?
--->Coven
The distance between your words is 9
The path between your words is : witch->watch->match->march->marcs-
>mares->cares->caves->coves->coven
*****************************************************************
Total execution time: 1495ms.
```

```
* Welcome to the Levenshtein Edit distance solver                *
* Please type in two words of the same size that will be used   *
* To find a path between them.                                   *
*****************************************************************

What word would you like to start with?
--->dog
What word would you like to end with?
--->CAT
The distance between your words is 3
The path between your words is : dog->cog->cot->cat
*****************************************************************
Total execution time: 38ms.
```

```
* Welcome to the Levenshtein Edit distance solver                *
* Please type in two words of the same size that will be used   *
* To find a path between them.                                   *
*****************************************************************

What word would you like to start with?
--->bookkeeper
What word would you like to end with?
--->sunglasses
No Path was found between your words.
*****************************************************************
Total execution time: 5081ms.
```

GETTING STARTED

In order to get started with this assignment, start by downloading the required files below into your project.

File	Actions
`dictionary.txt`	The primary list of words for the game to use.
`LevenDistanceFinderMain.java`	This is the file that you will run. No major modifications should be made to this file.

THE GENERAL ALGORITHM

From your initial list of words, compute a map from every word to its immediate neighbor words. A neighbor word is a word that is exactly the same except for one character is changed.

So DOG and DOT are neighbor words, as well as DOT and COT. But DOG and COT are not as they require two changes. Note that DOG is not a neighbor to DOG because the number of different letters is not 1 it is zero(0).

Once you have this map built, you can use it to find the distance between two words.

Then assuming the distance between two words is at least one, you can then find the link between two words.

YOUR INSTRUCTIONS

You are given a client program `LevenDistanceFinderMain.java` that does the file processing and user interaction. `LevenDistanceFinderMain.java` reads a dictionary text file as input and passes its entire contents to you as a list of strings. You are to write a class called `LevenshteinProcessor` that will manage the actual work of finding the distance and the path.

The class will probably have the following necessary private fields:

- Starting and Ending Strings
- A map of words to a set of neighbors.
- An integer distance that starts at -99.
    - Note that -1 means there is no path,
    - So -99 would mean that the distance hasn't been found yet.
- A List of strings that is the path itself.

NECESSARY METHODS

PUBLIC LEVENSHTEINPROCESSOR(STRING, STRING, SET<STRING>)

Your constructor is passed a string which is the starting word of the path, a string that is the ending word of the path, and a collection of all dictionary words as a set. It should use these values to initialize the class. The set of words will initially be all words from the dictionary.

You should throw an `IllegalArgumentException` if length of the two words is not the same or if the two words are the same.

You will want to store the starting and stopping words for later use, but you should not need to save the words list, instead you will create an internal list/set of words of the correct size for the starting ending words.

So, start by creating a list/set of words of correct size. Now, using this smaller list/set, what you will want to do is to create a map of words to their "neighbor" words. Start this by going through every word and add it and a new empty set, to a map.

Then you should then go through your word list/set a second time and check every word to every other word. If they are "neighbors" you will add each word to the set that goes with the other word. So if stringA and stringB are neigbors, then you add stringA to stringB's set, and vice versa.

Once the neighbor map is created, you should be done with the constructor.

PUBLIC INT GETDISTANCE()

This method is called by the main program to get the distance between the two words. If you haven't found the distance yet, run findDistance() otherwise you should just return a saved value.

PUBLIC STRING GETPATH()

This method returns a string that is the path from the first word to the second word, assuming it exists.

If the path distance is negative one, then return back a `null` reference. Otherwise, assuming the path distance is one or higher, then use `FindPath` (described below) to find the path list and convert it to a string with arrows in between.

Example return `love->lave->late->hate`

PRIVATE INT FINDDISTANCE(STRING A, STRING B)

This method finds the distance between two strings. *(Note, only the distance, not the path itself).* Start by creating two empty sets. Add the staring word to the second set. Define and set a counter to zero. Then while the two sets are different sizes **and** the 2nd set does not contain the ending word, add everything from set 2 into set one, clear set 2, and then take every word from set 1 AND the neighbor of every word from set1 into set 2. Increment the loop counter.

When the loop finishes, if the 2nd set contains the final word return the counter because that means a path was found, if the 2nd set doesn't contain the word, return -1 because there is no path.

Note, once you have found the distance, you should store it as a field of the class, so that in the future, if you call this method again, you can just return the value you found earlier, and skip actually running the long methods a second time.

PRIVATE LIST<STRING> FINDPATH(STRING A, STRING B)

This method will find the path between two words. It should probably be the last method you work on. In fact, I would create it, and have it return an empty List until you have everything else done.

When running this method should only be run after `findDistance()` has been run at least one. So if the private int value it is still at -99, it will need to run, but otherwise it should use the already stored value. If this distance is zero somehow, then it should throw an `IllegalStateException`.

When you are ready for it, this method should do the following process.

Initialize a list of string to new empty List. This will be the thing you need to return at the end.

Check the distance, if it is negative one, store the no path message in the path List and then return the list and exit the method.

If the distance is positive add the first element to the list.

Now in a loop that starts at the distance minus 1, and goes down until 1, look at the set of neighbors of the word in the first box of the list. Find one that has a distance to the ending word that matches the current loop counter. There may be multiples, but there has to be at least one. Find this word and add it to the front of the list.

Now repeat the loop until the for loop quits. Then add the ending word to the front of the list.

Reverse the list so it goes from first to last. You are now done and can return the list.

Here is an example for the path from `love -> hate`.

> The distance from love to hate is 3, so that is bigger than -1. So, add love to the list. The list is now size one. Now start your loop from distance -1 (2) to 1. So, i is currently 2. Find any word in the neighbor of love, that has a distance to hate of size 2. *Use your findDistance method here!*. One of those words is "lave". So add that to the array.

Next round, i is one. The list is now [love, lave]. So, find a neighbor of "lave" that has a distance to "hate" of one. One possible word is "late". So, add "late" to the list which now looks like [love, lave, late]

That should finish the loop, add "hate" to the list. The final list looks like [love, lave, late, hate]. Store that list in the class field. And you are done.

PRIVATE INT COUNTDIFFERENTLETTERS(STRING A, STRING B)

This method should take two strings and find the number of letters that are different from each other and return that number.

For example `countDifferentLetters("dog", "cat")` would return 3 because there are 3 different letters. While `countDifferentLetters ("tin", "tan")` would return 1 because they share the `'t'` and the `'n'`.

OTHER INSTRUCTIONS

Your program should exactly reproduce the format and general behavior demonstrated on the previous pages.

You may assume that the list of words passed to your constructor will be a nonempty list of nonempty strings composed entirely of lowercase letters.

Use the `TreeMaps, TreeSets` and `ArrayLists` implementations for all of your ADT's.

STYLE GUIDELINES AND GRADING:

You should introduce private methods to avoid redundancy and to break up large methods into smaller methods. In particular, **you should not have any methods that have more than 50 lines of code in their body** (not counting blank lines and lines that have just comments or curly braces). If you have a method that requires more than 50 lines of code, then you should break it up into smaller methods.

Make sure your output looks exactly like the sample output up above, you don't want to lose points because you decided to look a little different.

You may only use techniques and ideas that you have learned in this class. You are not allowed to use more advanced techniques that you might find on the internet. I want you to practice using the collection tools available as well as the idea of following a set plan.

You should follow general good style guidelines such as: making fields `private` and avoiding unnecessary fields; declaring collection variables using interface types; appropriately using control structures like loops and `if/else`; properly using indentation, good variable names and types; and not having any lines of code longer than 100 characters. Comment your code descriptively in your own words at the top of your class, each method, and on complex sections of your code. Comments should explain each method's behavior, parameters, return, pre/post-conditions, and exceptions.

For reference, one solution is around 150 lines long including comments and blank lines (100 "substantive" lines). However this is not a requirement, your code may be as long/short as you need it to be. But if you find yourself going excessively long, you may want to take a step back and reconsider your approach.

Note that this program may take some time to run. If you look at the examples above, “witch” and “coven” took 15 seconds to finish, and “bookkeeper”/“sunglasses” took even longer.

TESTING

The longest possible path is from “cabaret” to “atlases” which I will be testing. So make sure it works. It will take a while, this is normal but it should not take more than 90 seconds even on old machines.